Technische Universität Braunschweig

Project proposal

Artur Kück

Self-Imitation Learning for Robotic Cube Stacking

Objective and self-imitation hypothesis

Can a robot's final cube placement be improved through imitation of its own corrective movements? This question will be examined in a simulated task in which a third cube is placed on a two-cube tower. A slight misalignment may be corrected by a sideways movement, yet an otherwise stable tower may be disturbed by the same movement. The following hypothesis will be tested: a more reliable placement policy can be learned when recorded movements are weighted by their expected contribution to stacking success.

State of the art

In Self-Imitation Learning [1], an agent's past actions are reinforced when the observed return is higher than the estimated value of the state. In Critic Regularized Regression [2], imitation weights are derived from action values estimated on a fixed dataset. In Q-Planning [3], actions are selected at deployment with the aid of a critic while the robot policy is kept frozen. In the proposed study, short placement segments will be weighted by a critic during training. At deployment, commands will be generated by the trained policy alone.

From imitation learning to actor-critic

In behavior cloning (BC) for continuous control, an actor π_θ is fitted to observation–action pairs (x, a) from an expert dataset D_E. The mean squared difference between predicted and demonstrated commands is minimised:

$$L_{\mathrm{BC}}(\theta) = \mathbb{E}_{(x,a)\sim D_E}\left[\|\pi_\theta(x) - a\|_2^2\right]. \tag{1}$$

In an actor-critic method, the consequences of a command are estimated through an action-value function $Q_\phi(s, a)$. Its value is defined as the expected return when action a is taken in simulator state s and the policy is followed thereafter. For a deterministic actor [4], the critic and actor losses can be written as

$$y = r + \gamma(1 - d)\, Q_{\bar{\phi}}(s', \pi_{\bar{\theta}}(x')), \qquad L_Q(\phi) = \mathbb{E}[(Q_\phi(s, a) - y)^2], \tag{2}$$

$$L_{\mathrm{actor}}(\theta) = -\mathbb{E}[Q_\phi(s, \pi_\theta(x))]. \tag{3}$$

The reward is denoted by r, episode termination by d, and the next state or observation by a prime. Expectations in Eqs. (2)–(3) are estimated from replay data. The barred parameters are held fixed while the critic is fitted to target y. In Eq. (3), the actor is updated towards commands with higher predicted returns. In the proposed method, recorded commands will be scored by the critic and the actor will be trained by weighted imitation.

Proposed method and evaluation

A learning rule for critic-weighted self-imitation

A reference policy π_0 will be fitted to demonstrations and frozen. Placement segments τ will be recorded as bounded variations are applied to the robot's commands. In Eq. (2), π_0 will be used as the target actor. A reward of one will be assigned once for a successful stack and zero otherwise. With the remaining time included in s and $\gamma = 1$, the probability of success under the reference continuation can be estimated by the critic. For a segment of $H \leq 20$ steps, the score is defined as

$$S_\phi(\tau) = \sum_{t=0}^{H-1} \left[Q_\phi(s_t, a_t) - Q_\phi(s_t, \pi_0(x_t)) \right]. \tag{4}$$

At each visited state, the value of the reference command is subtracted from that of the executed command. A high, neutral or low positive weight w_τ will be assigned according to the score $S_\phi(\tau)$ and a preset tolerance around zero. The weights will then be normalised to mean one. In the combined loss, behavior-cloning terms are shown in blue and the critic-derived weight is shown in red.

$$L(\theta) = (1-\lambda)\underbrace{L_{\mathrm{BC}}(\theta)}_{\text{expert BC}} + \lambda\, \mathbb{E}_{\tau \sim D_{\mathrm{self}}} \left[\underbrace{w_\tau}_{\text{critic weight}} \underbrace{\frac{1}{H} \sum_{t=0}^{H-1} \|\pi_\theta(x_t) - a_t\|_2^2}_{\text{BC on the robot's actions}} \right]. \tag{5}$$

The robot's recorded segments are stored in D_{self}. Their contribution relative to expert demonstrations is controlled by $0 < \lambda < 1$. The weights are held fixed during actor training, so the imitation gradient for each segment is scaled by w_τ. Expert demonstrations are retained through the first term.

Research plan and evaluation

Each placement will be replayed from an identical simulator state. The sampled segment will be executed in one branch and reference commands in the other. Both trials will then be completed under π_0. A lost, unchanged or rescued stack will be coded as $\Delta Y = -1$, 0 or 1, respectively. Three actors will be trained with Eq. (5) using uniform weights, measured weights from ΔY or critic weights from Eq. (4). Data, initialisation, λ and update budget will be matched within each training seed. For each starting configuration, critic weights will be estimated by a model fitted on other configurations.
The three trained policies and π_0 will be evaluated on identical unseen starts across three training seeds. A trial will be counted as successful if the complete tower remains stable for 50 consecutive simulation steps. The critic's selections will be assessed by counting rescued and lost stacks. Policy improvement will be measured through paired differences in success rate.

References

[1] Oh et al. (2018). Self-Imitation Learning. ICML.
[2] Wang et al. (2020). Critic Regularized Regression. NeurIPS.
[3] Giridhar et al. (2026). Beyond Imitation: Self-Improving Robot Policies via Off-Policy Q-Planning. arXiv preprint.
[4] Silver et al. (2014). Deterministic Policy Gradient Algorithms. ICML.